



Advanced Graph Algorithms

Shortest Paths, MST, Topological Sort & More

Java 8+



Session Structure

1. Graph representations review
2. Shortest Path Algorithms
3. Minimum Spanning Tree
4. Topological Sorting
5. Strongly Connected Components
6. Union-Find
7. Real-world applications



Graph Representation (Quick Review)

Adjacency List:

```
java.util.List<java.util.List<Integer>> graph =  
    new java.util.ArrayList<>();
```

Time to build:

$O(V + E)$

Space:

$O(V + E)$



Dijkstra's Algorithm

Used for:

Shortest path in weighted graph (no negative weights).

Time:

$O(E \log V)$ with priority queue.



Dijkstra Code

```
static void dijkstra(java.util.List<java.util.List<int[]>> graph,
                    int src) {

    int n = graph.size();
    int[] dist = new int[n];
    java.util.Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    java.util.PriorityQueue<int[]> pq =
        new java.util.PriorityQueue<>(
            java.util.Comparator.comparingInt(a -> a[1]));

    pq.add(new int[]{src, 0});

    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int node = curr[0];
        int d = curr[1];

        if (d > dist[node]) continue;

        for (int[] neighbor : graph.get(node)) {
            int next = neighbor[0];
            int weight = neighbor[1];

            if (dist[node] + weight < dist[next]) {
                dist[next] = dist[node] + weight;
                pq.add(new int[]{next, dist[next]});
            }
        }
    }
}
```



Bellman-Ford Algorithm

Handles:

Negative weights.

Time:

$O(V * E)$

THIRAVUGAL

Bellman-Ford Code

```
static int[] bellmanFord(int V,  
    java.util.List<int[]> edges,  
    int src) {  
  
    int[] dist = new int[V];  
    java.util.Arrays.fill(dist, Integer.MAX_VALUE);  
    dist[src] = 0;  
  
    for (int i = 0; i < V - 1; i++) {  
        for (int[] e : edges) {  
            int u = e[0], v = e[1], w = e[2];  
            if (dist[u] != Integer.MAX_VALUE &&  
                dist[u] + w < dist[v])  
                dist[v] = dist[u] + w;  
        }  
    }  
  
    return dist;  
}
```



Floyd-Warshall Algorithm

All-pairs shortest path.

Time:

$O(V^3)$

Used for dense graphs.



Minimum Spanning Tree (MST)

Connect all vertices with:

Minimum total edge weight.



Prim's Algorithm

Time:
 $O(E \log V)$

THIRAVUGAL

Prim's Code (Simplified)

```
static void prim(java.util.List<java.util.List<int[]>> graph) {  
  
    int n = graph.size();  
    boolean[] visited = new boolean[n];  
  
    java.util.PriorityQueue<int[]> pq =  
        new java.util.PriorityQueue<>(  
            java.util.Comparator.comparingInt(a -> a[1]));  
  
    pq.add(new int[]{0, 0});  
  
    int totalCost = 0;  
  
    while (!pq.isEmpty()) {  
        int[] curr = pq.poll();  
        int node = curr[0];  
        int weight = curr[1];  
  
        if (visited[node]) continue;  
  
        visited[node] = true;  
        totalCost += weight;  
  
        for (int[] neighbor : graph.get(node)) {  
            if (!visited[neighbor[0]])  
                pq.add(neighbor);  
        }  
    }  
}
```



Kruskal's Algorithm

Sort edges → Add smallest edge without cycle.

Time:

$O(E \log E)$



Union-Find (Disjoint Set)

Used in:

Kruskal's algorithm.

Time:

Almost $O(1)$ with path compression.

Union-Find Code

```
static class DSU {
    int[] parent;
    int[] rank;

    DSU(int n) {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++)
            parent[i] = i;
    }

    int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]);
        return parent[x];
    }

    void union(int x, int y) {
        int px = find(x);
        int py = find(y);

        if (px == py) return;

        if (rank[px] < rank[py])
            parent[px] = py;
        else if (rank[px] > rank[py])
            parent[py] = px;
        else {
            parent[py] = px;
            rank[px]++;
        }
    }
}
```



Topological Sort

Used in:

Directed Acyclic Graph (DAG).

Time:

$O(V + E)$

THIRAVUGAL

Kahn's Algorithm

```
static void topoSort(int V,  
    java.util.List<java.util.List<Integer>> graph) {  
  
    int[] indegree = new int[V];  
  
    for (int i = 0; i < V; i++)  
        for (int neighbor : graph.get(i))  
            indegree[neighbor]++;  
  
    java.util.Queue<Integer> queue =  
        new java.util.LinkedList<>();  
  
    for (int i = 0; i < V; i++)  
        if (indegree[i] == 0)  
            queue.add(i);  
  
    while (!queue.isEmpty()) {  
        int node = queue.poll();  
  
        for (int neighbor : graph.get(node)) {  
            indegree[neighbor]--;  
            if (indegree[neighbor] == 0)  
                queue.add(neighbor);  
        }  
    }  
}
```




Strongly Connected Components

Kosaraju's Algorithm:

Time:

$O(V + E)$



Real World Use Cases

Routing systems:

Dijkstra.

Network design:

MST.

Build systems:

Topological sort.

Social networks:

SCC detection.

Distributed systems:

Union-Find for clustering.

Complexity Summary

Algorithm	Time
Dijkstra	$O(E \log V)$
Bellman-Ford	$O(VE)$
Floyd-Warshall	$O(V^3)$
Prim	$O(E \log V)$
Kruskal	$O(E \log E)$
Topological Sort	$O(V + E)$

Common Mistakes

- Using Dijkstra with negative weights
- Forgetting visited check
- Not handling disconnected graph
- Ignoring cycle detection
- Confusing MST with shortest path



Interview Strategy

Ask:

1. Is graph weighted?
2. Are weights negative?
3. Need single source or all pairs?
4. Directed or undirected?
5. Need cycle detection?



Performance Thought Exercise

Graph:

1 million nodes

10 million edges

$O(V^3)$ impossible.

Choose algorithm carefully.

Summary

Advanced graph algorithms power:

- Internet routing
- GPS navigation
- Network infrastructure
- Build systems
- Clustering

Master:

- Dijkstra
- Bellman-Ford
- MST (Prim & Kruskal)
- Topological sort
- Union-Find